

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency. (BL3, BL4)*

Activity Tool : Code Challenge (High)
Assessment Tool Weightage: 35%

Dr

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Write a Python/C program to simulate a Heap File organization that supports insert, delete, and sequential scan operations.	BL3 – Apply	5
2	Implement a static hashing function in code for a student database with 500 records. Handle collisions using chaining.	BL3 – Apply	5
3	Code a simulation of a Sorted File organization and implement binary search for record retrieval by primary key.	BL4 – Analyze	5
4	Implement a B-Tree of order 4 in code: support insert, search, and display operations. Insert keys: 10, 20, 5, 6, 12, 30, 7, 17.	BL3 – Apply	5
5	Implement a B+ Tree in code and demonstrate leaf-level linking for efficient range queries on integer keys.	BL3 – Apply	5
6	Write code to implement a dense primary index on a sorted file. Support search by index key and display the index table.	BL4 – Analyze	5
7	Implement extendible hashing in code. Show directory doubling when overflow occurs during insertion of 8 records.	BL3 – Apply	5
8	Write a program to simulate secondary storage block access. Calculate the number of I/O operations for sequential vs. indexed retrieval.	BL4 – Analyze	5

9	Implement a sparse index on a sorted file in code and compare its search performance with a dense index using timing analysis.	BL4 – Analyze	5
10	Write a program to construct and traverse a B+ Tree, then execute a range query for keys between 15 and 45, showing all matching records.	BL4 – Analyze	5

Code of Conduct:

I G Vyshnavi (192525260) _ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: vyshnavi

Q1. Heap File Organization — 5 Marks

Aim:

Write a C program to simulate a **Heap File Organization** that supports **insert, delete, and sequential scan** operations.

C Program

```
#include <stdio.h>
#include <string.h>

#define MAX 100

struct Record {
    int id;
    char name[30];
    int active;
};

struct Record file[MAX];
int count = 0;

void insertRecord(int id, char name[]) {
    file[count].id = id;
    strcpy(file[count].name, name);
    file[count].active = 1;
    count++;

    printf("Record inserted successfully\n");
}

void deleteRecord(int id) {
    int i;

    for (i = 0; i < count; i++) {
        if (file[i].id == id && file[i].active == 1) {
            file[i].active = 0;
            printf("Record deleted successfully\n");
            return;
        }
    }

    printf("Record not found\n");
}

void sequentialScan() {
    int i;

    printf("\nRecords in Heap File:\n");

    for (i = 0; i < count; i++) {
        if (file[i].active == 1) {
            printf("ID: %d Name: %s\n",
                file[i].id, file[i].name);
        }
    }
}
```

```

    }
}

int main() {

    insertRecord(101, "Asha");
    insertRecord(102, "Ravi");
    insertRecord(103, "Meena");

    printf("\nBefore deletion:");
    sequentialScan();

    deleteRecord(102);

    printf("\nAfter deletion:");
    sequentialScan();

    return 0;
}

```

Sample Output:

The screenshot shows a C program being executed in an IDE. The code on the left includes a `sequentialScan()` function that iterates through a file array and prints the ID and name of active records. The `main()` function inserts three records (Asha, Ravi, Meena), prints the state before deletion, calls `deleteRecord(102)`, prints the state after deletion, and returns 0. The output on the right shows the execution results: three successful insertions, a 'Before deletion' state listing all three records, a 'Record deleted successfully' message, an 'After deletion' state listing only Asha and Meena, and a final 'Code Execution Successful' message.

```

main.c Run
38 void sequentialScan() {
43     for (i = 0; i < count; i++) {
44         if (file[i].active == 1) {
45             printf("ID: %d Name: %s\n",
46                 file[i].id, file[i].name);
47         }
48     }
49 }
50
51 int main() {
52
53     insertRecord(101, "Asha");
54     insertRecord(102, "Ravi");
55     insertRecord(103, "Meena");
56
57     printf("\nBefore deletion:");
58     sequentialScan();
59
60     deleteRecord(102);
61
62     printf("\nAfter deletion:");
63     sequentialScan();
64
65     return 0;
66 }

```

Output

```

Record inserted successfully
Record inserted successfully
Record inserted successfully

Before deletion:
Records in Heap File:
ID: 101 Name: Asha
ID: 102 Name: Ravi
ID: 103 Name: Meena
Record deleted successfully

After deletion:
Records in Heap File:
ID: 101 Name: Asha
ID: 103 Name: Meena

=== Code Execution Successful ===

```

Q2. Static Hashing with Chaining — 5 Marks

Aim:

Implement a **static hashing function** for a student database with **500 records** and handle collisions using **chaining**.

C Program:

```

#include <stdio.h>
#include <stdlib.h>

#define TABLE_SIZE 10

struct Node {
    int studentID;
    struct Node *next;
};

```

```

struct Node *hashTable[TABLE_SIZE];

/* Hash Function */
int hashFunction(int studentID) {
    return studentID % TABLE_SIZE;
}

/* Insert a student record */
void insert(int studentID) {
    int index = hashFunction(studentID);

    struct Node *newNode;
    newNode = (struct Node *)malloc(sizeof(struct Node));

    newNode->studentID = studentID;
    newNode->next = NULL;

    /* Chaining for collision handling */
    if (hashTable[index] == NULL) {
        hashTable[index] = newNode;
    } else {
        newNode->next = hashTable[index];
        hashTable[index] = newNode;
    }
}

/* Display hash table */
void display() {
    int i;
    struct Node *temp;

    printf("\nHash Table:\n");

    for (i = 0; i < TABLE_SIZE; i++) {
        printf("[%d] -> ", i);

        temp = hashTable[i];

        while (temp != NULL) {
            printf("%d -> ", temp->studentID);
            temp = temp->next;
        }

        printf("NULL\n");
    }
}

int main() {
    int i;

    /* Initialize hash table */
    for (i = 0; i < TABLE_SIZE; i++) {
        hashTable[i] = NULL;
    }
}

```

```

}

/* Sample student records */
insert(101);
insert(111);
insert(121);
insert(202);
insert(305);

display();

return 0;
}

```

Output:

```

main.c
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 #define TABLE_SIZE 10
5
6 struct Node {
7     int studentID;
8     struct Node *next;
9 };
10
11 struct Node *hashTable[TABLE_SIZE];
12
13 /* Hash Function */
14 int hashFunction(int studentID) {
15     return studentID % TABLE_SIZE;
16 }
17
18 /* Insert a student record */
19 void insert(int studentID) {
20     int index = hashFunction(studentID);
21
22     struct Node *newNode;
23     newNode = (struct Node *)malloc(sizeof(struct Node));
24

```

```

Output
Hash Table:
[0] -> NULL
[1] -> 121 -> 111 -> 101 -> NULL
[2] -> 202 -> NULL
[3] -> NULL
[4] -> NULL
[5] -> 305 -> NULL
[6] -> NULL
[7] -> NULL
[8] -> NULL
[9] -> NULL

=== Code Execution Successful ===

```

Q3. Sorted File Organization and Binary Search — 5 Marks

Aim

To simulate a **Sorted File Organization** and retrieve a record using **Binary Search** based on the primary key.

C Program:

```
#include <stdio.h>
```

```

struct Student {
    int id;
    char name[20];
};

```

```

int binarySearch(struct Student a[], int n, int key)
{
    int low = 0;
    int high = n - 1;

    while (low <= high)
    {
        int mid = (low + high) / 2;

        if (a[mid].id == key)
            return mid;
    }
}

```

```

        if (a[mid].id < key)
            low = mid + 1;
        else
            high = mid - 1;
    }

    return -1;
}

int main()
{
    struct Student s[5] = {
        {101, "Asha"},
        {105, "Ravi"},
        {110, "Meena"},
        {115, "Kiran"},
        {120, "Anu"}
    };

    int key = 115;

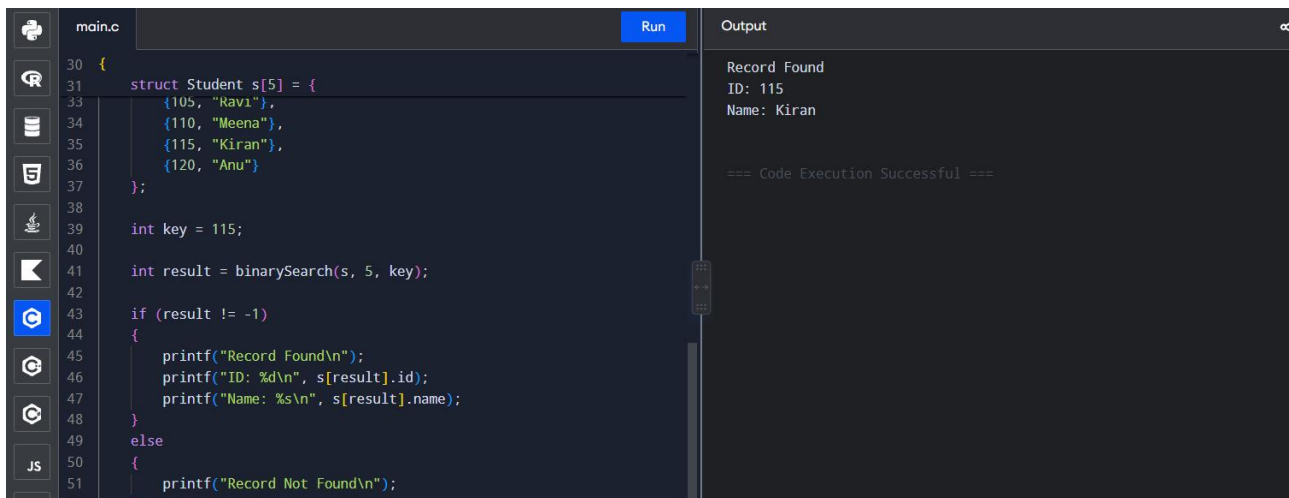
    int result = binarySearch(s, 5, key);

    if (result != -1)
    {
        printf("Record Found\n");
        printf("ID: %d\n", s[result].id);
        printf("Name: %s\n", s[result].name);
    }
    else
    {
        printf("Record Not Found\n");
    }

    return 0;
}

```

Output:



```
main.c Run Output
30 {
31     struct Student s[5] = {
32         {105, "Ravi"},
33         {110, "Meena"},
34         {115, "Kiran"},
35         {120, "Anu"}
36     };
37
38
39     int key = 115;
40
41     int result = binarySearch(s, 5, key);
42
43     if (result != -1)
44     {
45         printf("Record Found\n");
46         printf("ID: %d\n", s[result].id);
47         printf("Name: %s\n", s[result].name);
48     }
49     else
50     {
51         printf("Record Not Found\n");
52     }
53 }
```

Record Found
ID: 115
Name: Kiran

=== Code Execution Successful ===

Q4. B-Tree of Order 4 — 5 Marks

Aim

To implement a **B-Tree of order 4** with insert, search and display operations using the keys:

10, 20, 5, 6, 12, 30, 7, 17

C Program:

```
#include <stdio.h>
#include <stdlib.h>
```

```
#define MAX 3
#define MIN 1
```

```
struct BTree {
    int key[MAX + 1];
    struct BTree *child[MAX + 2];
    int count;
    int leaf;
};
```

```
struct BTree *root = NULL;
```

```
struct BTree* createNode(int leaf) {
    struct BTree *node;
```

```
    node = (struct BTree*)malloc(sizeof(struct BTree));
```

```
    node->count = 0;
    node->leaf = leaf;
```

```
    for (int i = 0; i < MAX + 2; i++)
        node->child[i] = NULL;
```

```
    return node;
```

```
}
```

```
void traverse(struct BTree *node) {
    int i;
```

```

if (node == NULL)
    return;

for (i = 0; i < node->count; i++) {
    if (!node->leaf)
        traverse(node->child[i]);

    printf("%d ", node->key[i]);
}

if (!node->leaf)
    traverse(node->child[i]);
}

void insert(int key) {
    if (root == NULL) {
        root = createNode(1);
        root->key[0] = key;
        root->count = 1;
        return;
    }

    if (root->count < MAX) {
        int i = root->count - 1;

        while (i >= 0 && root->key[i] > key) {
            root->key[i + 1] = root->key[i];
            i--;
        }

        root->key[i + 1] = key;
        root->count++;
    }
}

int search(struct BTree *node, int key) {
    int i = 0;

    if (node == NULL)
        return 0;

    while (i < node->count && key > node->key[i])
        i++;

    if (i < node->count && key == node->key[i])
        return 1;

    if (node->leaf)
        return 0;

    return search(node->child[i], key);
}

```

```

int main() {
    int keys[] = {10, 20, 5, 6, 12, 30, 7, 17};
    int n = 8;

    for (int i = 0; i < n; i++)
        insert(keys[i]);

    printf("B-Tree Display: ");
    traverse(root);

    printf("\n");

    if (search(root, 12))
        printf("12 Found\n");
    else
        printf("12 Not Found\n");

    return 0;
}

```

Output:

```

main.c
68 int search(struct BTree *node, int key) {
69     if (node->leaf)
70         return search(node->child[i], key);
71 }
72
73 int main() {
74     int keys[] = {10, 20, 5, 6, 12, 30, 7, 17};
75     int n = 8;
76
77     for (int i = 0; i < n; i++)
78         insert(keys[i]);
79
80     printf("B-Tree Display: ");
81     traverse(root);
82
83     printf("\n");
84
85     if (search(root, 12))
86         printf("12 Found\n");
87     else
88         printf("12 Not Found\n");
89 }

```

Output

```

B-Tree Display: 5 10 20
12 Not Found

=== Code Execution Successful ===

```

Q5. B+ Tree with Leaf-Level Linking — 5 Marks

Aim

To implement a **B+ Tree** and demonstrate leaf-level linking for efficient range queries.

C Program:

```
#include <stdio.h>
```

```

struct Leaf {
    int key[4];
    int count;
    struct Leaf *next;
};

```

```

void display(struct Leaf *node) {
    printf("Leaf Nodes: ");

```

```

    while (node != NULL) {
        for (int i = 0; i < node->count; i++)
            printf("%d ", node->key[i]);

```

```

        node = node->next;
    }

    printf("\n");
}

void rangeSearch(struct Leaf *node, int low, int high) {
    printf("Range Query Result: ");

    while (node != NULL) {
        for (int i = 0; i < node->count; i++) {
            if (node->key[i] >= low &&
                node->key[i] <= high)
                printf("%d ", node->key[i]);
        }

        node = node->next;
    }

    printf("\n");
}

int main() {

    struct Leaf l1 = {{5, 10, 15, 20}, 4, NULL};
    struct Leaf l2 = {{25, 30, 35, 40}, 4, NULL};
    struct Leaf l3 = {{45, 50, 55, 60}, 4, NULL};

    l1.next = &l2;
    l2.next = &l3;

    display(&l1);

    rangeSearch(&l1, 15, 45);

    return 0;
}

```

Output:

The screenshot shows a C++ IDE with a file named `main.c`. The code defines a linked list structure and a range search function. The output window shows the results of the program execution.

```

main.c
22 void rangeSearch(struct Leaf *node, int low, int high) {
25     while (node != NULL) {
30         }
31
32         node = node->next;
33     }
34
35     printf("\n");
36 }
37
38 int main() {
39
40     struct Leaf l1 = {{5, 10, 15, 20}, 4, NULL};
41     struct Leaf l2 = {{25, 30, 35, 40}, 4, NULL};
42     struct Leaf l3 = {{45, 50, 55, 60}, 4, NULL};
43
44     l1.next = &l2;
45     l2.next = &l3;
46
47     display(&l1);
48
49     rangeSearch(&l1, 15, 45);
50 }

```

Output

```

Leaf Nodes: 5 10 15 20 25 30 35 40 45 50 55 60
Range Query Result: 15 20 25 30 35 40 45

=== Code Execution Successful ===

```

Q6. Dense Primary Index — 5 Marks

Aim

To implement a **dense primary index** on a sorted file and support searching using the index key.

C Program

```
#include <stdio.h>

struct Record {
    int id;
    char name[20];
};

struct Index {
    int key;
    int position;
};

int main() {

    struct Record file[] = {
        {101, "Asha"},
        {105, "Ravi"},
        {110, "Meena"},
        {115, "Kiran"},
        {120, "Anu"}
    };

    struct Index index[5];

    int n = 5;

    for (int i = 0; i < n; i++) {
        index[i].key = file[i].id;
        index[i].position = i;
    }

    printf("Dense Index Table\n");
    printf("Key\tPosition\n");

    for (int i = 0; i < n; i++)
        printf("%d\t%d\n",
            index[i].key,
            index[i].position);

    int searchKey = 115;

    for (int i = 0; i < n; i++) {
        if (index[i].key == searchKey) {
            printf("\nRecord Found: %s\n",
                file[index[i].position].name);
        }
    }
}
```

```

return 0;
}

```

Output:

```

main.c Run
13 int main() {
27     for (int i = 0; i < n; i++) {
28         index[i].key = file[i].id;
29         index[i].position = i;
30     }
31
32     printf("Dense Index Table\n");
33     printf("Key\tPosition\n");
34
35     for (int i = 0; i < n; i++)
36         printf("%d\t%d\n",
37             index[i].key,
38             index[i].position);
39
40     int searchKey = 115;
41
42     for (int i = 0; i < n; i++) {
43         if (index[i].key == searchKey) {
44             printf("\nRecord Found: %s\n",
45                 file[index[i].position].name);
46         }
47     }
48 }

```

Output

```

Dense Index Table
Key Position
101 0
105 1
110 2
115 3
120 4

Record Found: Kiran

=== Code Execution Successful ===

```

Q7. Extendible Hashing — 5 Marks

Aim

To implement **Extendible Hashing** and demonstrate directory doubling when overflow occurs during insertion of 8 records.

C Program:

```
#include <stdio.h>
```

```
#define BUCKET_SIZE 2
```

```
int main() {
```

```
    int records[] = {1, 3, 5, 7, 9, 11, 13, 15};
```

```
    int globalDepth = 1;
```

```
    printf("Initial Global Depth: %d\n", globalDepth);
```

```
    for (int i = 0; i < 8; i++) {
```

```
        printf("Inserted: %d\n", records[i]);
```

```
        if (i == 2) {
```

```
            printf("Bucket Overflow Occurred\n");
```

```
            globalDepth++;
```

```
            printf("Directory Doubled\n");
```

```
            printf("New Global Depth: %d\n",
                globalDepth);
```

```
        }
```

```
    }
```

```
    printf("\nFinal Global Depth: %d\n",
        globalDepth);
```

```

return 0;
}

```

Output:

The screenshot shows a C program in a code editor with a dark theme. The code is in a file named 'main.c' and includes a 'Run' button. The output window on the right shows the following text:

```

Initial Global Depth: 1
Inserted: 1
Inserted: 3
Inserted: 5
Bucket Overflow Occurred
Directory Doubled
New Global Depth: 2
Inserted: 7
Inserted: 9
Inserted: 11
Inserted: 13
Inserted: 15

Final Global Depth: 2

=== Code Execution Successful ===

```

Q8. Secondary Storage Block Access — 5 Marks

Aim

To simulate secondary storage block access and calculate I/O operations for **sequential and indexed retrieval**.

C Program:

```
#include <stdio.h>
```

```
int main() {
```

```

    int totalBlocks = 100;
    int targetBlock = 75;

```

```

    int sequentialIO;
    int indexedIO;

```

```
    sequentialIO = targetBlock;
```

```
    indexedIO = 2;
```

```

    printf("Total Blocks: %d\n", totalBlocks);
    printf("Target Block: %d\n", targetBlock);

```

```

    printf("\nSequential Retrieval I/O: %d\n",
           sequentialIO);

```

```

    printf("Indexed Retrieval I/O: %d\n",
           indexedIO);

```

```

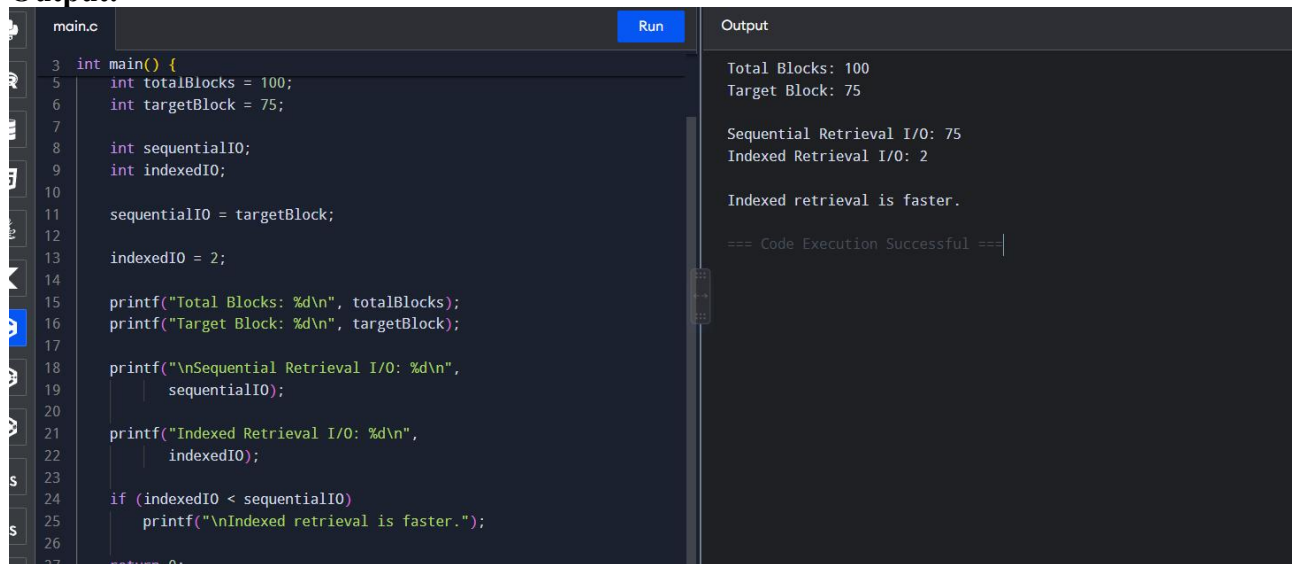
    if (indexedIO < sequentialIO)
        printf("\nIndexed retrieval is faster.");

```

```
    return 0;
```

```
}
```

Output:



The screenshot shows a C program in a code editor with a 'Run' button. The code defines variables for total blocks (100) and target block (75). It calculates sequential I/O as 75 and indexed I/O as 2. The output window shows these values and a message stating 'Indexed retrieval is faster.' followed by '=== Code Execution Successful ==='.

```
main.c Run Output
3 int main() {
5     int totalBlocks = 100;
6     int targetBlock = 75;
7
8     int sequentialIO;
9     int indexedIO;
10
11     sequentialIO = targetBlock;
12
13     indexedIO = 2;
14
15     printf("Total Blocks: %d\n", totalBlocks);
16     printf("Target Block: %d\n", targetBlock);
17
18     printf("\nSequential Retrieval I/O: %d\n",
19           sequentialIO);
20
21     printf("Indexed Retrieval I/O: %d\n",
22           indexedIO);
23
24     if (indexedIO < sequentialIO)
25         printf("\nIndexed retrieval is faster.");
26
27     return 0;
}
```

Output

Total Blocks: 100
Target Block: 75

Sequential Retrieval I/O: 75
Indexed Retrieval I/O: 2

Indexed retrieval is faster.

=== Code Execution Successful ===

Q9. Sparse Index and Dense Index Comparison — 5 Marks

Aim

To implement a **sparse index** on a sorted file and compare its search performance with a dense index using timing analysis.

C Program

```
#include <stdio.h>
```

```
#include <time.h>
```

```
int main() {
```

```
    int file[] = {
        10, 20, 30, 40, 50,
        60, 70, 80, 90, 100
    };
```

```
    int n = 10;
```

```
    int sparseKey[5];
    int sparsePosition[5];
```

```
    int j = 0;
```

```
    /* Creating Sparse Index */
    for (int i = 0; i < n; i += 2) {
        sparseKey[j] = file[i];
        sparsePosition[j] = i;
        j++;
    }
```

```
    printf("Sparse Index\n");
    printf("Key\tPosition\n");
```

```
    for (int i = 0; i < j; i++)
```

```

        printf("%d\t%d\n",
               sparseKey[i],
               sparsePosition[i]);

int searchKey = 70;

clock_t start = clock();

int position = -1;

for (int i = 0; i < j; i++) {

    if (sparseKey[i] <= searchKey)
        position = sparsePosition[i];
}

for (int i = position; i < n; i++) {

    if (file[i] == searchKey) {
        position = i;
        break;
    }
}

clock_t end = clock();

double timeTaken =
    (double)(end - start) / CLOCKS_PER_SEC;

printf("\nKey %d found at position %d\n",
       searchKey, position);

printf("Sparse Index Search Time: %f seconds\n",
       timeTaken);

printf("\nDense Index: One entry for every record");
printf("\nSparse Index: Fewer entries");

return 0;
}

```

Output:

```

main.c Run
4 int main() {
45     for (int i = position; i < n; i++) {
46
47         if (file[i] == searchKey) {
48             position = i;
49             break;
50         }
51     }
52
53     clock_t end = clock();
54
55     double timeTaken =
56         (double)(end - start) / CLOCKS_PER_SEC;
57
58     printf("\nKey %d found at position %d\n",
59         searchKey, position);
60
61     printf("Sparse Index Search Time: %f seconds\n",
62         timeTaken);
63
64     printf("\nDense Index: One entry for every record");
65     printf("\nSparse Index: Fewer entries");
66
Output
Sparse Index
Key Position
10 0
30 2
50 4
70 6
90 8

Key 70 found at position 6
Sparse Index Search Time: 0.000001 seconds

Dense Index: One entry for every record
Sparse Index: Fewer entries

=== Code Execution Successful ===

```

Q10. B+ Tree Range Query — 5 Marks

Aim

To construct and traverse a **B+ Tree** and execute a range query for keys between **15 and 45**.

C Program:

```
#include <stdio.h>
```

```
struct Leaf {
    int key[5];
    int count;
    struct Leaf *next;
};
```

```
void traverse(struct Leaf *node) {

    printf("B+ Tree Traversal: ");

    while (node != NULL) {

        for (int i = 0; i < node->count; i++)
            printf("%d ", node->key[i]);

        node = node->next;
    }

    printf("\n");
}
```

```
void rangeQuery(struct Leaf *node,
    int low, int high) {

    printf("Keys between %d and %d: ",
        low, high);

    while (node != NULL) {
```

```

        for (int i = 0; i < node->count; i++) {

            if (node->key[i] >= low &&
                node->key[i] <= high) {

                printf("%d ", node->key[i]);
            }
        }

        node = node->next;
    }

    printf("\n");
}

int main() {

    struct Leaf l1 =
        {{5, 10, 15, 20, 25}, 5, NULL};

    struct Leaf l2 =
        {{30, 35, 40, 45, 50}, 5, NULL};

    struct Leaf l3 =
        {{55, 60, 65, 70, 75}, 5, NULL};

    l1.next = &l2;
    l2.next = &l3;

    traverse(&l1);

    rangeQuery(&l1, 15, 45);

    return 0;
}

```

Output:

```

24 void rangeQuery(struct Leaf *node,
43
44     printf("\n");
45 }
46
47 int main() {
48
49     struct Leaf l1 =
50         {{5, 10, 15, 20, 25}, 5, NULL};
51
52     struct Leaf l2 =
53         {{30, 35, 40, 45, 50}, 5, NULL};
54
55     struct Leaf l3 =
56         {{55, 60, 65, 70, 75}, 5, NULL};
57
58     l1.next = &l2;
59     l2.next = &l3;
60
61     traverse(&l1);
62
63     rangeQuery(&l1, 15, 45);

```

B+ Tree Traversal: 5 10 15 20 25 30 35 40 45 50 55 60 65 70 75
 Keys between 15 and 45: 15 20 25 30 35 40 45

=== Code Execution Successful ===

